

AI / ML Interview Prep

Complete Question & Answer Guide

For AI Engineers, GenAI Engineers & ML Roles

49 questions across 8 topics | Medium → Hard level | Interviewer-ready answers

Topics Covered

1. Machine Learning (12 questions)
2. CNN (4 questions)
3. Transformers & LLMs (8 questions)
4. RAG (Retrieval-Augmented Generation) (9 questions)
5. Embeddings & Vector Databases (4 questions)
6. System Design (4 questions)
7. FastAPI Backend (3 questions)
8. Coding / DSA (5 questions)

Q1. Why does Linear Regression use MSE instead of MAE?

MSE is differentiable everywhere, making gradient descent optimization smooth. MAE has a non-differentiable point at zero. MSE also penalizes large errors more heavily (squaring), forcing the model to reduce big mistakes aggressively.

Interviewer tip: MSE = differentiable everywhere, smooth gradients, outlier sensitivity. Always mention these 3.

Q2. What assumptions does Linear Regression make?

1. Linearity — relationship between X and Y is linear.
2. Independence — observations are independent.
3. Homoscedasticity — constant variance of residuals.
4. Normality — residuals are normally distributed.
5. No Multicollinearity — features are not highly correlated with each other.

Interviewer tip: Remember L-I-H-N-M: Linearity, Independence, Homoscedasticity, Normality, No multicollinearity.

Q3. What is multicollinearity and how do you detect it?

Multicollinearity occurs when two or more independent variables are highly correlated. This makes it hard to isolate individual variable effects — coefficients become unstable and standard errors inflate.

Detection: (1) VIF (Variance Inflation Factor) — VIF > 5 is concerning, VIF > 10 is serious. (2) Correlation matrix — pairwise correlation > 0.8. (3) Condition Number from eigenvalues.

Interviewer tip: Always mention VIF first. It's the standard answer interviewers expect.

Q4. What is heteroscedasticity?

Heteroscedasticity means the variance of residuals is not constant — it changes across levels of the independent variable. Violates the homoscedasticity assumption.

Detection: Plot residuals vs fitted values — a funnel/cone shape is the signal.

Fix: Log-transform the target, use weighted least squares, or robust standard errors.

Interviewer tip: Key phrase: 'non-constant variance in residuals.'

Q5. Batch vs Mini-Batch vs Stochastic Gradient Descent?

Batch GD: Uses all training data per step. Stable but very slow for large datasets.

Mini-Batch GD: Uses a subset (32-256 samples). Best of both worlds — faster than Batch, more stable than SGD. This is the standard in practice.

Stochastic GD: Uses one sample per step. Fast but noisy, can miss the minimum.

Interviewer tip: Always say 'mini-batch is the standard in practice' — interviewers love that.

Q6. What is Ridge vs Lasso Regression?

Ridge (L2): Loss = $\text{MSE} + \lambda \times \sum(w^2)$. Shrinks coefficients toward zero but never to exactly zero. Good for multicollinearity, retains all features.

Lasso (L1): Loss = $\text{MSE} + \lambda \times \sum|w|$. Can shrink coefficients to exactly zero — automatic feature selection. Good for high-dimensional data with irrelevant features.

Interviewer tip: L2 = Ridge = shrinks but keeps features. L1 = Lasso = zeros out features. Easy to remember.

Q7. Why do Decision Trees overfit? How does Random Forest fix it?

Decision Trees overfit because they keep splitting until each leaf has one sample — memorizing noise. No built-in regularization.

Random Forest fixes this with: (1) Bagging — each tree trains on a random data subset, reducing variance. (2) Feature randomness — only a random subset of features considered per split, decorrelating trees. Averaging many diverse, uncorrelated trees cancels out individual errors.

Interviewer tip: Say: 'Bagging + random feature selection = diverse uncorrelated trees. Averaging reduces variance.'

Q8. Explain XGBoost architecture.

XGBoost is a sequential ensemble where each new tree corrects residual errors of previous trees.

Key innovations: (1) Second-order Taylor expansion — uses gradient AND hessian for better optimization. (2) Built-in L1+L2 regularization. (3) Parallel split evaluation. (4) 'Gamma' pruning threshold. (5) Automatic handling of missing values.

Interviewer tip: Highlight: second-order derivatives, built-in regularization, parallel computation. That's what makes XGBoost special.

Q9. Vanishing Gradient problem and solutions?

During backpropagation, gradients multiply at each layer. With sigmoid activations (output 0-1), gradients shrink exponentially in early layers — they barely update.

Solutions: (1) ReLU activation — no saturation for positive values. (2) Batch Normalization — normalizes activations. (3) Residual connections (ResNets) — gradient highways bypass layers. (4) Gradient clipping. (5) Better initialization (He, Xavier).

Interviewer tip: Cause: sigmoid saturates → gradients shrink. Fix: ReLU, BatchNorm, residual connections.

Q10. Why ReLU? What is the Dying ReLU problem?

ReLU = $\max(0, x)$. Popular because: no vanishing gradient for positive values, computationally cheap, sparse activation (many neurons output 0), and trains faster empirically.

Dying ReLU: If a neuron's input is always negative, it always outputs 0. Gradient is 0, so no updates ever happen — the neuron is permanently dead.

Fix: Leaky ReLU (small negative slope), ELU, or GELU (used in Transformers).

Interviewer tip: Dying ReLU = neuron stuck at 0, gradient is 0, never recovers. Fix = Leaky ReLU.

Q11. What is Batch Normalization?

BatchNorm normalizes layer inputs by subtracting the batch mean and dividing by batch std dev, then applying learnable scale (γ) and shift (β) parameters.

Benefits: (1) Reduces internal covariate shift. (2) Allows higher learning rates — more stable training. (3) Mild regularization effect. (4) Reduces sensitivity to weight initialization.

Interviewer tip: Key phrase: 'reduces internal covariate shift.' Interviewers love that term.

Q12. What is Dropout?

Dropout randomly sets a fraction (20-50%) of neuron outputs to zero during training. Different neurons are dropped each step.

Why it works: Forces redundant representations — no single neuron can memorize a pattern because it might be absent. Like training an ensemble of many subnetworks and averaging at test time.

At inference: all neurons active, outputs scaled by keep probability.

Interviewer tip: 'Forces redundant representations — like training an ensemble of subnetworks.' Great framing.

CNN

Q1. Why convolution instead of fully connected layers?

Convolution exploits spatial structure: (1) Parameter sharing — the same filter detects the same pattern anywhere in the image. (2) Local connectivity — each neuron only sees a small region. This massively reduces parameters and prevents overfitting. A 1000x1000 image fully connected = 1B parameters; CNN uses ~millions.

Interviewer tip: Say: 'translation invariance via parameter sharing + local connectivity = efficient and powerful.'

Q2. CNN key concepts: kernel, stride, padding, pooling, feature maps

Kernel/Filter: Small matrix slid over input to detect a pattern (edges, textures). Kernel size (3x3, 5x5) controls how large a region it sees.

Stride: How many pixels the filter moves per step. Larger stride = smaller output.

Padding: Adding zeros around the input border — 'same' padding maintains output size.

Max Pooling: Takes the maximum value in each region — preserves dominant features, reduces spatial size.

Feature Map: The output of applying a filter to the input — shows where that pattern was detected.

Interviewer tip: The flow: Input → Convolution (feature extraction) → Pooling (downsampling) → FC layers (classification).

Q3. What is Transfer Learning? Why does it work?

Transfer Learning: Take a model pretrained on a large dataset (e.g., ImageNet) and fine-tune it on your smaller, specific dataset.

Why it works: Early CNN layers learn universal features — edges, textures, shapes — that are useful across many visual tasks. You reuse these learned representations instead of training from scratch.

Approach: Freeze early layers, retrain the final layers on your task. If your dataset is very different, unfreeze more layers.

Interviewer tip: 'Early layers = universal features. Later layers = task-specific. Fine-tune accordingly.'

Q4. What are ResNets and why do skip connections work?

ResNets (Residual Networks) add skip connections — shortcuts that bypass one or more layers: $\text{output} = F(x) + x$.

Why they work: (1) Gradient highway — gradients flow directly back through the skip connection, solving vanishing gradients in very deep networks. (2) Identity mapping is easy to learn — if a layer isn't needed, it just learns $F(x) = 0$, and the skip connection passes x through. Enabled training of 100+ layer networks.

Interviewer tip: 'Skip connections are gradient highways that allow training of very deep networks without vanishing gradients.'

Transformers & LLMs

Q1. Why did Transformers replace RNNs?

RNNs process sequences step-by-step — can't parallelize training. Also suffer vanishing gradients with long sequences — they 'forget' early tokens.

Transformers solve both: (1) Self-attention processes all tokens simultaneously — fully parallelizable on GPUs. (2) Direct connections between any two tokens regardless of distance — no forgetting. Tradeoff: $O(n^2)$ memory complexity with sequence length.

Interviewer tip: Key points: parallelization + direct long-range dependencies. RNNs fail at both.

Q2. Explain self-attention mathematically.

Self-attention computes a weighted sum of all tokens based on relevance to the current token.

1. Create Q (Query), K (Key), V (Value) by multiplying input X by learned weight matrices.
2. Compute scores: $\text{score} = Q \times K^T$
3. Scale: divide by $\sqrt{d_k}$ to stabilize gradients
4. Softmax: $\text{weights} = \text{softmax}(QK^T / \sqrt{d_k})$
5. Output: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$

Interviewer tip: Formula to memorize: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$

Q3. Why Query, Key, Value?

The Q, K, V framework is from information retrieval:

Query (Q): What the current token is looking for.

Key (K): What each token advertises about itself.

Value (V): The actual information to aggregate.

YouTube analogy: Query = your search term. Keys = video titles/tags. Values = the actual video content. You match query to keys, then retrieve values.

Interviewer tip: Use the YouTube/database analogy. Interviewers remember it.

Q4. Why scale attention scores by $\sqrt{d_k}$?

The dot product $Q \times K^T$ gets very large when embedding dimension d_k is large. Large values push softmax into regions with extremely small gradients — causing vanishing gradients.

Dividing by $\sqrt{d_k}$ keeps variance of dot products at roughly 1 regardless of dimension size, keeping softmax well-behaved and training stable.

Interviewer tip: 'Large dot products push softmax to saturation. $\sqrt{d_k}$ scaling stabilizes gradients.'

Q5. What is Multi-head attention?

Instead of one attention function, multi-head attention runs h attention functions in parallel, each with its own Q, K, V projections.

Why: Different heads can attend to different relationship types simultaneously — syntax, coreference, semantic similarity, etc.

Process: Run h heads → concatenate outputs → project with final weight matrix W_o . Like an ensemble of attention functions.

Interviewer tip: 'Multiple heads learn different relationship types in parallel.' Mention it's like ensemble of attention.

Q6. Encoder vs Decoder in Transformers?

Encoder: Reads the full input using bidirectional self-attention (every token attends to every other). For understanding tasks. Example: BERT.

Decoder: Generates tokens one at a time using masked (causal) self-attention — only attends to previous tokens. For generation tasks. Example: GPT.

Encoder-Decoder: Encoder reads input, decoder generates output while attending to encoder via cross-attention. For translation, summarization. Example: T5.

Interviewer tip: BERT = encoder (understands). GPT = decoder (generates). T5 = encoder-decoder.

Q7. How are LLMs trained? Explain RLHF.

Phase 1 — Pretraining: Train on trillions of tokens using next-token prediction. Model learns language, facts, reasoning patterns.

Phase 2 — SFT (Supervised Fine-Tuning): Fine-tune on high-quality prompt-response pairs to follow instructions.

Phase 3 — RLHF: (1) Collect human preference rankings of model outputs. (2) Train a Reward Model on these rankings. (3) Fine-tune LLM using PPO to maximize reward model score, with KL penalty to prevent drift.

Interviewer tip: Three phases: Pretraining → SFT → RLHF. Know each one. Mention KL penalty for depth.

Q8. What is quantization?

Quantization reduces model precision — storing weights in lower bit formats (INT8, INT4, INT2) instead of FP32/FP16.

Benefits: 4-bit = 8x smaller than FP32, faster inference (integer ops faster than float ops), lower memory — allows running large models on consumer hardware.

Techniques: GPTQ, AWQ minimize accuracy loss. QLoRA uses 4-bit quantization for fine-tuning large models on consumer GPUs.

Interviewer tip: 'INT4 quantization makes a 70B model runnable on 2x A100 instead of 8x.' Great concrete example.

RAG (Retrieval-Augmented Generation)

Q1. Explain the complete RAG pipeline.

Indexing phase (offline): Load documents → Chunk into segments → Embed each chunk → Store in vector DB.

Retrieval phase (online): User query → Embed query → ANN Search in vector DB → Optional re-ranking → Top-k chunks.

Generation phase: Inject chunks into LLM prompt as context → LLM generates answer grounded in context. This solves hallucination by giving the model real, relevant facts to work with.

Interviewer tip: Structure as 3 phases: Indexing → Retrieval → Generation. Interviewers want a systematic answer.

Q2. RAG vs Fine-Tuning — when to use which?

RAG: Best for up-to-date or frequently changing information, domain-specific documents, explainability (can cite sources). No training needed. Knowledge lives outside the model — easy to update.

Fine-Tuning: Best when you need to change the model's behavior, tone, or output format. Internalizes patterns deeply. Requires training data and compute.

Key rule: Use RAG for knowledge, Fine-Tuning for behavior/style. In production, combining both is often most powerful.

Interviewer tip: 'RAG = knowledge injection. Fine-tuning = behavior change.' Interviewers love clean distinctions.

Q3. Why is chunking needed? Best chunk size? What is chunk overlap?

Chunking is needed because LLMs have context window limits — you can't embed an entire document meaningfully as one vector.

Too small: Chunks lack context, answers are incomplete.

Too large: Embeddings become noisy, retrieval less precise.

Sweet spot: 256-512 tokens — always benchmark for your specific documents.

Chunk overlap (50-100 tokens): Ensures context isn't cut off at boundaries — critical for coherent answers across chunk boundaries.

Interviewer tip: Say 'it depends on document type, and I'd benchmark 256-512 tokens.' Never claim one size fits all.

Q4. What is hybrid search? BM25 vs Vector Search?

Hybrid search combines two retrieval methods:

Vector search (semantic): Embedding similarity — finds semantically similar chunks even if exact words differ.

BM25 (lexical): Classic TF-IDF based ranking — great for exact matches, specific terminology, product codes.

Why combine: Vector search misses exact keyword matches. BM25 misses paraphrased queries. Hybrid catches both.

Merge results with Reciprocal Rank Fusion (RRF).

Interviewer tip: 'Semantic + lexical = hybrid. Vector for meaning, BM25 for keywords. Combine with RRF.'

Q5. What is re-ranking and why does it improve results?

After initial ANN retrieval (top-k chunks), a re-ranker model scores each chunk specifically for relevance to the query and reorders them.

Why needed: Embedding similarity is a fast approximation — top-k may not be in the best order.

Re-rankers use cross-encoding (process query + chunk together), giving much more accurate relevance scores. Slower but only applied to the small top-k set.

Models: Cohere Rerank, bge-reranker.

Interviewer tip: 'Bi-encoder (fast, approximate) → cross-encoder reranker (slow, accurate). Two-stage retrieval.'

Q6. How do you evaluate RAG quality?

Retrieval metrics:

- Context Precision — of retrieved chunks, how many are actually relevant?
- Context Recall — of all relevant chunks, how many were retrieved?

Generation metrics:

- Faithfulness — is the answer grounded in retrieved context? (no hallucination)
- Answer Relevance — does the answer address the question?

Framework: RAGAS automates these metrics using an LLM as judge. Also use a human-evaluated golden QA dataset as ground truth.

Interviewer tip: RAGAS: Faithfulness, Answer Relevance, Context Precision, Context Recall. Memorize these 4.

Q7. What is semantic chunking?

Traditional chunking splits by fixed token count — can cut a paragraph mid-thought. Semantic chunking uses embeddings to detect natural topic boundaries.

How it works: Embed consecutive sentences. If cosine similarity between adjacent sentences drops significantly, that's a topic boundary — split there.

Advantage: Each chunk is more self-contained and semantically coherent, leading to better retrieval quality. Implemented in LangChain's SemanticChunker.

Interviewer tip: 'Split where semantics change, not where a character count hits.' Clear, memorable.

Q8. What causes hallucinations? How to reduce them?

Root causes: (1) Training objective is next-token prediction — optimized for fluency, not factuality. (2) Knowledge gaps — if facts weren't in training data, model interpolates. (3) Over-confidence — no calibrated sense of what it doesn't know. (4) Long-tail knowledge — rare facts seen fewer times.

Mitigation: RAG (ground responses in documents), chain-of-thought reasoning, explicit uncertainty prompting, RLHF alignment, Constitutional AI.

Interviewer tip: 'Optimized for fluency, not factuality.' That's the root cause — interviewers love this framing.

Q9. What is Chain of Thought? ReAct? Self-Consistency?

Chain of Thought (CoT): Ask the model to reason step-by-step before answering ('Let's think step by step'). Breaks complex problems into checkable steps. Zero-shot CoT needs no examples.

ReAct: Interleaves reasoning (Thought) and actions (Act) — model thinks, then calls a tool, then thinks again. Used for agents.

Self-Consistency: Sample multiple reasoning paths (with temperature > 0), take the majority answer. More reliable than a single chain of thought.

Interviewer tip: Three named techniques: CoT (reasoning), ReAct (reasoning + actions), Self-Consistency (ensemble reasoning).

Embeddings & Vector Databases

Q1. How are embeddings generated?

Word embeddings (Word2Vec): Two shallow networks (CBOW or Skip-Gram) predict words from context. The weight matrix becomes the embedding.

Sentence embeddings (SBERT): Text passes through a BERT-like encoder; mean-pooling over token outputs gives a single fixed-size vector.

Key insight: Semantically similar texts appear in similar contexts during training → their vectors end up close in the high-dimensional space.

Interviewer tip: Mention SBERT for production sentence embeddings — shows you know modern practice, not just Word2Vec.

Q2. Cosine similarity vs Euclidean distance for embeddings?

Cosine similarity measures the angle between vectors, ignoring magnitude. Range: -1 to 1. Two vectors pointing the same direction = 1.

Euclidean distance measures straight-line distance — sensitive to vector magnitude.

Why cosine is preferred: Embeddings encode meaning in direction, not magnitude. A long document might have a larger norm without being 'more similar' to anything. Cosine correctly focuses on direction (semantic content).

Interviewer tip: 'Embeddings encode meaning in direction, not magnitude — cosine ignores magnitude, so it's preferred.'

Q3. What is ANN Search? Explain HNSW.

ANN (Approximate Nearest Neighbor): Finds close vectors without checking every vector — exact search is $O(n)$, too slow at scale.

HNSW (Hierarchical Navigable Small World): Builds a multi-layer graph. Top layers have long-range sparse connections (coarse navigation). Bottom layers have dense local connections (precise matching).

Search starts at top (fast, coarse) and navigates down to precise neighbors — like zooming in on a map. Used in FAISS, Qdrant, Weaviate.

Interviewer tip: 'Multi-layer graph — coarse to fine navigation.' The map analogy is great for interviews.'

Q4. What is embedding drift?

Embedding drift occurs when stored vectors no longer align with embeddings your current model generates — e.g., after upgrading the embedding model.

Problem: Queries embedded with the new model are compared against old-model vectors — similarity scores become meaningless, retrieval quality degrades silently.

Solution: Re-embed all documents when upgrading models. Monitor retrieval quality metrics. Version your embedding models.

Interviewer tip: 'Upgrading your embedding model invalidates your entire index — you must re-embed everything.'

System Design

Q1. Design a RAG System at scale.

Ingestion Pipeline: Document loaders (PDF, web, DB) → Chunking (512 tokens, 50 overlap) → Embedding model (SBERT/OpenAI) → Vector DB (Pinecone/Qdrant/pgvector).

Query Pipeline: Query → Embed → ANN Search → Reranker → Top-k chunks → Prompt assembly → LLM → Response.

Infrastructure: FastAPI backend, Redis cache for frequent queries, async ingestion, monitoring with LangSmith/RAGAS.

Scalability: Horizontal scaling of API servers, distributed vector DB, CDN, rate limiting.

Guardrails: Input validation, faithfulness check, citation of source chunks.

Interviewer tip: Structure: Ingestion → Query → Generation. Then mention caching, monitoring, and guardrails.'

Q2. Design an AI Assistant (like ChatGPT) for 1M users.

Frontend: React/Flutter with WebSocket for streaming responses.

API Gateway: JWT auth, rate limiting, load balancing.

Conversation Service: FastAPI, maintains history, summarizes old messages when context window fills.

LLM Service: Streaming. Fallback to secondary model on failure.

Memory: Short-term = conversation buffer. Long-term = vector store of past conversations.

Storage: PostgreSQL for user data, Redis for session cache.

Scaling: Auto-scaling groups, async processing, semantic caching (GPTCache), token budgets per user.

Interviewer tip: Must mention: streaming, context management, memory, fallback, and scaling. Hit all 5.

Q3. How would you reduce latency in an AI app?

Model-level: Quantization (INT4/INT8), smaller/distilled models, speculative decoding.

Infrastructure: Stream responses token-by-token (don't wait for full response), GPU inference servers (vLLM, TGI) with batching, CDN/edge deployment, semantic caching (GPTCache) — cache responses for similar queries.

Application: Reduce context length (compress history), parallel tool calls, pre-compute embeddings offline.

Interviewer tip: Group answer: Model-level → Infrastructure → Application. Shows systematic thinking.

Q4. Design a Real-Time Chat Application.

Connection: WebSocket for bidirectional real-time messaging.

Message Service: FastAPI/Node.js WebSocket server. Messages published to Kafka/Redis Streams.

Storage: Cassandra or DynamoDB for messages (write-heavy). PostgreSQL for user/group metadata.

Delivery: Queue consumers push to online users via WebSocket. Offline users get push notifications (FCM/APNs).

Scale: Consistent hashing to route users to same WebSocket server. Redis pub/sub for cross-server messaging.

Features: Read receipts, message ordering (Lamport timestamps), end-to-end encryption.

Interviewer tip: Key: WebSockets + message queue (Kafka) + separate storage for messages vs metadata.

FastAPI Backend

Q1. Async vs Sync in FastAPI?

Sync (def): Runs in a thread pool. If the function blocks (DB query, file read), it occupies a thread. Many concurrent requests can exhaust the thread pool.

Async (async def): Runs in the event loop. While waiting for I/O, the event loop handles other requests — no thread is occupied during the wait.

Rule: Use async def with async libraries (asyncpg, httpx). Use def for CPU-bound tasks or legacy sync libraries — FastAPI auto-runs them in a thread pool.

Critical mistake to avoid: Calling a blocking sync library from async def — this blocks the entire event loop.

Interviewer tip: 'async for I/O-bound, sync for CPU-bound. Never use blocking code inside async def.'

Q2. JWT Authentication in FastAPI?

Flow: (1) User sends credentials to /login. (2) Server verifies, creates JWT (header.payload.signature in base64). (3) Client stores JWT. (4) Every request sends JWT in Authorization header: 'Bearer '. (5) Server verifies signature using secret key, extracts user from payload.

In FastAPI: Use python-jose for JWT, passlib for password hashing. Token validation as a Depends() dependency.

Security best practices: Short expiry access tokens (15-60 min) + longer refresh tokens. Always HTTPS.

Interviewer tip: Mention access token + refresh token pattern. Shows you understand production auth.

Q3. What is Dependency Injection in FastAPI?

FastAPI's Depends() system lets you declare shared logic (auth, DB sessions, config) as dependencies that FastAPI automatically resolves and injects into route handlers.

Benefits: (1) Reusable across routes. (2) Easy to override in tests. (3) Clean lifecycle management — DB sessions open and close correctly. (4) Hierarchical — dependencies can have their own dependencies.

Interviewer tip: 'Depends() = declarative, testable, reusable logic injection. Like Spring DI but Pythonic.'

Coding / DSA

Q1. Two Sum — O(n) solution

Approach: Hash map. For each number, check if (target - num) already exists in the map. If yes, return both indices. If no, add the number to the map.

def twoSum(nums, target):

seen = {}

for i, n in enumerate(nums):

diff = target - n

if diff in seen:

return [seen[diff], i]

seen[n] = i

Time: O(n), **Space:** O(n)

Interviewer tip: Always start with brute force O(n^2), then optimize to O(n). Shows your thought process.

Q2. Maximum Subarray — Kadane's Algorithm

Insight: At each position, decide — extend the current subarray or start fresh here.

def maxSubArray(nums):

max_sum = curr_sum = nums[0]

for n in nums[1:]:

curr_sum = max(n, curr_sum + n)

max_sum = max(max_sum, curr_sum)

return max_sum

Time: O(n), **Space:** O(1)

Interviewer tip: 'At each step: extend or restart?' That's the core insight of Kadane's.

Q3. BFS and DFS

BFS — uses a queue, explores level by level, good for shortest path in unweighted graphs:

```
from collections import deque
```

```
def bfs(graph, start):
```

```
    visited, queue = {start}, deque([start])
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        for nb in graph[node]:
```

```
            if nb not in visited:
```

```
                visited.add(nb); queue.append(nb)
```

DFS — uses stack/recursion, explores deep first, good for cycle detection, topological sort, connected components.

Interviewer tip: BFS = queue = shortest path. DFS = stack/recursion = cycles, topological sort.

Q4. Coin Change — Dynamic Programming

DP: $dp[i]$ = minimum coins to make amount i . Try every coin for each amount.

```
def coinChange(coins, amount):
```

```
    dp = [float('inf')] * (amount + 1)
```

```
    dp[0] = 0
```

```
    for a in range(1, amount + 1):
```

```
        for c in coins:
```

```
            if c <= a:
```

```
                dp[a] = min(dp[a], 1 + dp[a - c])
```

```
    return dp[amount] if dp[amount] != float('inf') else -1
```

Time: $O(\text{amount} \times \text{coins})$, **Space:** $O(\text{amount})$

Interviewer tip: Classic unbounded knapsack. 'For each amount, try all coins.' Base case: $dp[0] = 0$.

Q5. Longest Common Subsequence

DP: If characters match, extend previous LCS. If not, take max of skipping one from either string.

```
def lcs(s1, s2):
```

```
    m, n = len(s1), len(s2)
```

```
    dp = [[0] * (n+1) for _ in range(m+1)]
```

```
    for i in range(1, m+1):
```

```
        for j in range(1, n+1):
```

```
            if s1[i-1] == s2[j-1]:
```

```
                dp[i][j] = 1 + dp[i-1][j-1]
```

```
            else:
```

```
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

```
    return dp[m][n]
```

Time: $O(m \times n)$, **Space:** $O(m \times n)$

Interviewer tip: Draw the DP table on a whiteboard. Visualizing it makes the recurrence obvious.

Good luck with your interview!

Practice each answer out loud. The goal is to explain clearly and confidently — not to recite.